

AutoTabber

Audio to Beginner Guitar Tablature

Course Project: Final Submission



Team Members:

Daniel Ahn

Shauryaditya Singh

GitHub: github.com/shauryadata/autotabberV1

Live Demo: huggingface.co/spaces/shauryadata/autotabberv1

Python 3.12 | Streamlit | Demucs | Basic-pitch | librosa | SQLite

1. Project Description

AutoTabber is a Python web application that takes an audio file and converts it into beginner-friendly guitar tablature. The idea came from the frustration of trying to learn songs on guitar without having access to accurate tabs. Most tabs online are user-submitted and often wrong, and commercial transcription software tends to be expensive and overly complex for someone just picking up the instrument. We wanted to build something that could listen to a song and produce a simple, playable tab that a beginner could actually follow.

The app accepts MP3, WAV, and M4A audio files. Once a file is uploaded, the user can optionally enable source separation to isolate a specific instrument from the recording before pitch detection runs. After that, the user picks one of two pitch detection backends. The first is pYIN, a classical algorithm from librosa that tracks one note at a time. It works well on clean recordings with a single instrument. The second is Basic-pitch, a convolutional neural network developed by Spotify that can detect multiple notes playing at the same time. This means it can handle chords and full-band recordings, which is a big deal because most real-world music is polyphonic.

After pitch detection, the raw note data goes through several processing stages. A monophonic melody filter collapses near-simultaneous detections into a clean single-line tab. A note filtering step prunes detections by amplitude, duration, and frequency range to remove noise and artifacts. A beat-grid quantizer snaps the notes to musically meaningful time slots (quarter notes, eighth notes, or sixteenth notes depending on user preference). A fretboard mapper then figures out where each note should be played on the guitar neck, prioritizing open strings and lower fret positions because those are easier for beginners. The final output is a standard six-line ASCII guitar tab with bar lines every sixteen columns, a header showing tempo and duration, and a legend at the bottom. The user can read it on screen, download it as a text file, or access it later through a built-in tab history backed by a SQLite database.

The entire interface runs through Streamlit, which gives us a clean browser-based GUI without needing to deal with frontend frameworks. All the audio processing, source separation, pitch detection, quantization, and fretboard mapping happens on the backend in Python. The app is deployed on both Streamlit Cloud (lightweight, pYIN only) and Hugging Face Spaces (full power, with Basic-pitch and Demucs source separation), so anyone with a browser can try it out without installing anything locally.

2. Project Requirements

Below is a walkthrough of how AutoTabber satisfies each requirement from the project specification. This section has been updated to reflect everything added since the milestone submission, including source separation, expanded testing, and dual deployment.

2.1 Language and Python Version

The entire application is written in Python. Our local development environment runs Python 3.12.10, and the Hugging Face Spaces deployment uses Python 3.12 as well. The Streamlit Cloud deployment runs the same code on its own Python build. All code uses type hints and features available in Python 3.12.

2.2 GitHub Repository

Our code is hosted at github.com/shauryadata/autotabberV1 on the main branch. Both team members have made substantial commits throughout development, covering the addition of new classes, source separation integration, pitch detection refinements, expanded test coverage, GUI improvements, deployment configuration, and documentation updates.

2.3 Classes

AutoTabber has eight distinct classes, each handling a specific part of the pipeline. Every class is stored in its own .py file inside the `beginner_tab` package and imported into `app.py` as a module. The new `SourceSeparator` class was added during the source separation phase to handle Demucs preprocessing.

Class	File	Purpose
AudioLoader	audio_loader.py	Loads MP3/WAV/M4A files, converts to WAV via pydub, returns a normalized mono NumPy array at 22,050 Hz
SourceSeparator	source_separator.py	Optional Demucs preprocessing that splits audio into 4 stems (vocals/drums/bass/other) and returns the user-selected stem
PitchTracker	pitch_tracker.py	Monophonic pitch detection using librosa pYIN with optional HPSS preprocessing
BasicPitchTracker	pitch_tracker.py	Polyphonic pitch detection using Spotify Basic-pitch with ONNX/TensorFlow backend
TabSimplifier	tab_simplifier.py	Quantizes raw note events onto a beat grid and collapses sustained notes
FretboardMapper	fretboard_mapper.py	Maps MIDI notes to guitar string/fret positions using a beginner-friendly scoring system
TabRenderer	tab_renderer.py	Renders mapped positions as a standard six-line ASCII guitar tab with bar lines and metadata header
TabStorage	tab_storage.py	CRUD operations for saving, listing, viewing, and deleting tabs in a SQLite database

2.4 Multiple .py Files

The project is organized into a main application file, a package of modules, a scripts directory, and a test suite:

- `app.py`: Streamlit frontend that ties everything together
- `beginner_tab/__init__.py`: Package initializer that re-exports all classes and probes for optional features
- `beginner_tab/audio_loader.py`: `AudioLoader` class
- `beginner_tab/source_separator.py`: `SourceSeparator` class (added in Phase 2)
- `beginner_tab/pitch_tracker.py`: `PitchTracker` and `BasicPitchTracker` classes
- `beginner_tab/tab_simplifier.py`: `TabSimplifier` class
- `beginner_tab/fretboard_mapper.py`: `FretboardMapper` class
- `beginner_tab/tab_renderer.py`: `TabRenderer` class
- `beginner_tab/tab_storage.py`: `TabStorage` class
- `scripts/verify_source_separator.py`: Standalone smoke test for the Demucs separator
- `tests/`: Five test modules covering the pipeline end to end

Each class is imported from its module into `app.py` through the package's `__init__.py`, keeping the main application file clean and readable. Optional classes (`SourceSeparator`, `BasicPitchTracker`) are exposed conditionally through availability probes so the package still imports cleanly on systems where the heavy dependencies are not installed.

2.5 Graphical User Interface

The GUI is built with Streamlit, which renders as a full browser-based web application. Users interact entirely through the browser. There is no terminal interaction or text-based interface required. The layout includes a sidebar for settings and a main content area for results.

2.6 Interactive GUI Elements

The application has well over five interactive elements. The list below has grown since the milestone with the addition of source separation controls, monophonic mode, the note filtering sliders, and the difficulty mode selector:

#	Element	What It Does
1	File Uploader	Drag and drop or browse for an MP3, WAV, or M4A audio file
2	Source Separation Toggle	Optional checkbox that routes audio through Demucs before pitch detection (local only)
3	Stem Selector	Picks which Demucs stem (vocals/drums/bass/other) to transcribe when separation is on
4	Detector Radio Button	Selects between Basic-pitch (polyphonic AI) and pYIN (monophonic) pitch detection
5	Monophonic Melody Mode	Checkbox that collapses near-simultaneous detections into a single-line melody
6	Note Filtering Sliders	Amplitude threshold and minimum note duration sliders to prune noise and artifacts

#	Element	What It Does
7	Difficulty Mode Selector	Beginner (max fret 5) / Intermediate (max fret 7) / Advanced (max fret 12)
8	Note Grid Selectbox	Sets the quantization resolution: quarter, eighth, or sixteenth notes
9	One-String Checkbox	Restricts output to the high-e string only for absolute beginners
10	Columns Per Line Slider	Controls how many note columns appear per line in the tab output
11	Generate Tab Button	Runs the full pipeline: load, separate (optional), detect, filter, quantize, map, render
12	Download Button	Downloads the generated tab as a .txt file with metadata header
13	Advanced Tuning Sliders	Onset, frame, and minimum note length controls for Basic-pitch; HPSS and confidence for pYIN
14	Tab History Controls	View, download, or delete previously generated tabs from the SQLite history

2.7 Advanced Python Modules

Our primary advanced module is TensorFlow, which powers the Basic-pitch neural network for polyphonic pitch detection. Basic-pitch is a convolutional neural network trained by Spotify on a large dataset of real music recordings. It takes raw audio as input and outputs timestamped MIDI note events, including chords. Internally, the model runs through the ONNX Runtime backend for faster inference, but the model itself was trained and exported using TensorFlow.

PyTorch was added in Phase 2 as the second advanced ML framework. It powers Demucs, Meta's open source source separation model. Demucs takes a full-mix audio recording and splits it into four isolated stems: vocals, drums, bass, and other (which contains guitar, keys, and most other pitched instruments). When source separation is enabled, only the user-selected stem is fed into pitch detection, which dramatically reduces transcription noise on real-world recordings.

We also make heavy use of librosa for the pYIN pitch detection algorithm, Harmonic-Percussive Source Separation (HPSS), beat tracking, and frequency-to-MIDI conversion. SQLite (via Python's built-in sqlite3 module) handles all persistent data storage. The TabStorage class manages a local database where every generated tab is saved automatically with metadata including the detector used, tempo, max fret setting, the stem source (full mix or one of the four Demucs stems), and a UTC timestamp.

2.8 Data I/O

AutoTabber performs meaningful data I/O in two directions:

- **Input:** The AudioLoader class reads audio files from disk. For non-WAV formats (MP3, M4A), it uses pydub with ffmpeg to decode and convert the audio into a temporary WAV file. librosa then loads this WAV as a NumPy array at 22,050 Hz mono. When source separation is enabled, the SourceSeparator class additionally writes a temporary WAV containing only the chosen Demucs stem, which then becomes the new input to the pitch tracker.
- **Output and storage:** Generated tabs are saved to a SQLite database file (tabs.db) through the TabStorage class. The app performs full CRUD operations: creating new

records on tab generation, reading them for the history view, updating the schema through an idempotent migration that adds new columns to existing databases, and deleting records at the user's request. Users can also download tabs as .txt files with a metadata header.

2.9 Testing with pytest

We have five test files in the tests/ directory covering the core pipeline components. The test count grew significantly during Phase 1 and Phase 2 as new functionality was added:

- `test_fretboard_mapper.py`: Tests scoring rules, position lookup, max-fret enforcement, one-string mode behavior, and the same-string continuation logic
- `test_tab_renderer.py`: Tests bar line placement, fixed-width column formatting, two-digit fret alignment, MIDI-to-note-name conversion, tab structure (six strings, correct labels), and line wrapping
- `test_tab_simplifier.py`: Tests slot duration math, quantization accuracy, range simplification, and sustained-note collapse
- `test_chord_pipeline.py`: End-to-end tests for the polyphonic pipeline including chord mapping, chord rendering, monophonic melody extraction, octave snapping in the aggressive filter pipeline, and TabStorage CRUD operations
- `test_source_separator.py`: Tests the SourceSeparator module load path, the available stems list, the `is_available` probe, and the invalid-stem error path. These tests skip cleanly when Demucs is not installed, so the suite continues to pass on environments without the heavy ML dependencies.

The full suite contains 118 passing tests (114 from the milestone-era pipeline plus 4 new source separator tests). All tests use synthetic data and in-memory SQLite databases so they run in seconds without requiring audio files or model downloads.

2.10 Code Documentation

Every .py file has a header comment block that includes the authors, date, and a description of what the file does. All classes and public methods have docstrings explaining their purpose, arguments, and return values. Key variables and logic blocks are annotated with inline comments. New code added in Phase 2 follows the same convention. For example, the `source_separator.py` file documents the choice to use the canonical `demucs.pretrained.get_model` API path (rather than the `Separator` class which does not exist in the current shipping version of Demucs), and the choice to bypass `demucs.audio.AudioFile` in favor of `soundfile` and `librosa` for the audio loading step. The `fretboard_mapper.py` file documents every scoring rule and the rationale behind each weight.

2.11 User Instructions

The app includes a collapsible "How to Use AutoTabber" section at the top of the main page. It walks the user through the steps of uploading a file, optionally enabling source separation, picking a detector, adjusting settings, clicking Generate Tab, and downloading the result. Every sidebar control has a help tooltip that appears on hover. The README on GitHub is comprehensive and includes a Features table, two install paths (lightweight cloud-style versus full local), a Usage section that documents every sidebar control, a Quality Tips section with practical advice on getting clean output, and a dedicated Source Separation Mode section explaining when and how to use the optional Demucs preprocessing stage.

3. Final Submission

This section covers the current state of AutoTabber for the final submission. The project has expanded substantially since the milestone, with two major development phases (Phase 1 quality improvements and Phase 2 source separation) and a second deployment target on Hugging Face Spaces.

3.1 Functionality

The final version of AutoTabber delivers the following working features:

- **Audio Loading:** Users can upload MP3, WAV, or M4A files. The AudioLoader class handles format conversion and normalization. The app validates file size, checks for silence, and warns about very short or very long recordings.
- **Source Separation (new in Phase 2):** When the user enables the Source Separation toggle, the audio runs through Spotify's open source Demucs model before pitch detection. Demucs produces four stems (vocals, drums, bass, other) and the user picks which one to transcribe. The Other stem is the right choice for guitar transcription on full-band recordings; the Vocals stem captures the sung melody. This stage adds 30 to 90 seconds of processing time but dramatically reduces the noise that pitch detection sees.
- **Pitch Detection:** Both detection backends are functional. pYIN handles single-note melodies with optional HPSS preprocessing. Basic-pitch handles chords and polyphonic audio through its ONNX-backed neural network. Both apply guitar-range frequency bandpass and MIDI range filtering.
- **Note Filtering Pipeline (new in Phase 1):** Detected notes pass through a five-step cleaning pipeline before quantization. Notes outside the guitar MIDI range (40 to 88) are dropped. Notes below the user-tunable amplitude threshold are dropped. Notes shorter than the user-tunable minimum duration are dropped. Notes above MIDI 76 are octave-snapped down to keep tabs in beginner-friendly territory. Consecutive same-pitch detections within 100 ms are merged into single longer notes.
- **Monophonic Melody Mode (new in Phase 1):** An optional post-filter that bins notes into 50 ms windows and keeps only the loudest note in each window. This collapses polyphonic detections into a clean single-line melody, which is what most beginners want to learn first. It can be disabled when the user explicitly wants Basic-pitch chord output.
- **Beat Quantization:** Raw note events are snapped to a musical grid based on the detected tempo. The user can select quarter, eighth, or sixteenth note resolution. Sustained notes held across adjacent beats are collapsed into single events.
- **Smart Fretboard Mapping (rewritten in Phase 1):** MIDI notes are placed on the guitar fretboard using a scoring system rather than a fixed lookup. Open strings get +100 points. Lower frets get up to +20 points. Frets above the user's difficulty cap get -1000 points (effectively excluded). Big jumps from the previous note get -30 points. Continuing on the same string as the previous note gets +10 points. The combination with the highest total score wins. The user picks one of three difficulty modes (Beginner caps at fret 5, Intermediate at 7, Advanced at 12).
- **Tab Rendering (rewritten in Phase 1):** The output is a six-line ASCII guitar tab with a metadata header showing tempo, total notes, duration, tuning, max fret, and source stem. Bar lines appear every sixteen columns (one measure of sixteenth notes). Each

note column is exactly three characters wide so single- and double-digit frets stay aligned. A footer legend explains what each row and number means.

- **Data Persistence:** Every generated tab is automatically saved to a SQLite database with metadata including the source stem. The Tab History section lets users browse, view, download, or delete past results. An idempotent schema migration handles upgrading older databases from the milestone era to the new schema without data loss.
- **Deployment (expanded in Phase 2):** The app is live on two platforms. The Streamlit Cloud build (autotabber.streamlit.app) runs a lightweight pYIN-only configuration because Streamlit Cloud's free tier cannot host the full ML dependency stack. The Hugging Face Spaces build (huggingface.co/spaces/shauryadata/autotabberv1) runs the full version including Basic-pitch and Demucs source separation thanks to its 16 GB RAM allocation and Docker-based Python control.

3.2 User Interactions

A typical session on the full version looks like this: the user opens the app, uploads a guitar recording, enables Source Separation, picks the Other stem, selects Basic-pitch as the detector, sets the difficulty to Beginner, and clicks Generate Tab. The app shows a status spinner during separation (30 to 90 seconds), then a second status block during the rest of the pipeline. When it finishes, the tab appears on screen with a success message showing the number of events, the estimated tempo, and the source stem used. The user can then download the tab as a .txt file or find it later in the Tab History section.

If the user wants to experiment, they can change the detector, swap stems, adjust the amplitude and duration thresholds, try a different difficulty mode, and regenerate. Each run produces a new entry in the history with full metadata, so the user can compare results side by side.

3.3 Files Included

The following files are included in the repository and are required to execute the project:

File	Description
app.py	Main application file, Streamlit frontend
beginner_tab/__init__.py	Package initializer with availability probe for Basic-pitch and Demucs
beginner_tab/audio_loader.py	AudioLoader class for file loading
beginner_tab/source_separator.py	SourceSeparator class wrapping Demucs (optional, local only)
beginner_tab/pitch_tracker.py	PitchTracker and BasicPitchTracker classes
beginner_tab/tab_simplifier.py	TabSimplifier class for beat quantization
beginner_tab/fretboard_mapper.py	FretboardMapper class with scoring-based fret placement
beginner_tab/tab_renderer.py	TabRenderer class for ASCII output with bar lines and metadata
beginner_tab/tab_storage.py	TabStorage class for SQLite persistence with stem column and migrations
scripts/verify_source_separator.py	Standalone smoke test for the Demucs separator

File	Description
tests/test_chord_pipeline.py	End-to-end tests for the chord pipeline plus cross-pipeline assertions
tests/test_fretboard_mapper.py	Unit tests for fretboard scoring and placement
tests/test_tab_renderer.py	Unit tests for ASCII rendering, bar lines, and column widths
tests/test_tab_simplifier.py	Unit tests for beat quantization
tests/test_source_separator.py	Unit tests for the Demucs separator (skips when Demucs is not installed)
requirements.txt	Cloud deployment dependencies (lightweight, no Demucs or PyTorch)
requirements-local.txt	Full local development dependencies including Demucs and PyTorch
packages.txt	System packages for Streamlit Cloud (ffmpeg)

3.4 How to Run

AutoTabber has two install paths depending on which features the user wants:

For the lightweight install (pYIN detector only, no source separation, ~50 MB of dependencies):

- Clone the repository: `git clone https://github.com/shauryadata/autotabberV1.git`
- Create and activate a virtual environment: `python3 -m venv venv && source venv/bin/activate`
- Install dependencies: `pip install -r requirements.txt`
- Make sure ffmpeg is installed on the system
- Launch the app: `streamlit run app.py`

For the full local install (everything above plus Basic-pitch and Demucs, ~1 GB of dependencies):

- Same first two steps as above
- Install dependencies: `pip install -r requirements-local.txt`
- Launch the app: `streamlit run app.py`

To run the full test suite: `pytest tests/ -v`

To exercise the source separator in isolation: `python scripts/verify_source_separator.py`

Both deployments are also available without any local install. Streamlit Cloud (autotabber.streamlit.app) runs the lightweight build. Hugging Face Spaces (huggingface.co/spaces/shauryadata/autotabberv1) runs the full build with Basic-pitch and Demucs.

3.5 Known Limitations

- Basic-pitch requires Python 3.12 or earlier because of its TensorFlow dependency. There are no pre-built TensorFlow wheels for Python 3.13 yet.

- Demucs source separation adds 30 to 90 seconds of processing per song on CPU. The first run on any machine downloads roughly 80 MB of model weights.
- Generated tabs represent note sequence only. Rhythm durations (how long each note is held) are not encoded in the ASCII output.
- Pitch detection accuracy depends heavily on the quality and complexity of the input audio. Source separation helps significantly on full-band recordings, but cannot fully recover from heavily compressed or distorted input.
- The fretboard mapper optimizes for hand position comfort using its scoring system, not for musical phrasing. Some fingerings may differ from how a skilled guitarist would play the passage.
- Streamlit Cloud cannot host the full version because of resource limits. The lightweight pYIN-only build is provided there as a fallback.

3.6 What Changed Since the Milestone

The milestone submission delivered a working pipeline that produced tabs from audio files. The final submission builds on that foundation in three meaningful directions.

Phase 1 focused on tab quality. The note filtering pipeline, monophonic melody mode, smart fretboard scoring system, and rewritten tab renderer all came out of this phase. The result is tabs that are noticeably cleaner, easier to play, and visually closer to what a beginner expects from a tab they would find online.

Phase 2 focused on tackling the biggest source of pitch detection error on real-world recordings: the fact that vocals, drums, bass, and guitar all overlap in the input audio. The new SourceSeparator class wraps Demucs and lets the user transcribe just one stem at a time. This single addition produces dramatically better tabs on full-band recordings than any amount of threshold tuning could achieve.

Deployment expanded to a second platform. The original Streamlit Cloud build remains live, but a new Hugging Face Spaces build provides the full feature set thanks to its larger free-tier resource limits and Docker-based Python control. Users can now choose between the two depending on which features they need.

Test coverage expanded from the milestone-era 42 tests to 118 tests, all passing.

Documentation expanded with a comprehensive README covering features, install paths, usage, quality tips, source separation guidance, project layout, and known limitations.